

# Exerciții — secvențe și frecvențe

Fișă de exerciții · vectori, C++ · continuare la `frecventa.h` și `secvente.h`

**Ce ai voie să folosești:** `int`, `double`, `bool`; vectori `int v[100]` cu dimensiunea ținută separat în `d`; funcții cu parametri (`int v[]`, `int d`) și cu `int& dim` când dimensiunea se schimbă; `for`, `while`, `do...`, `while`, `if/else`, `break`; operatorii / și %; vector de frecvență `int f[10]{} / int f[101]{}; sqrt` din `<cmath>`. Ai voie (și e recomandat) să refolosești funcțiile pe care le-ai scris deja: `prim`, `sumaCifrelor`, `numarCifre`, `oglindit`, `cmmdc`, `afisare`, `frecventaCif`.

**Reține — cele patru reguli din review:** (1) dimensiunea unui vector de frecvență = cea mai mare valoare posibilă + 1; (2) garda se pune *înaintea* accesului: `while (i < d-1 && v[i] == v[i+1])`, nu invers; (3) `d` trebuie să fie exact câte elemente ai scris între acolade; (4) după fiecare modificare, **rulează cu F5** și compară cu răspunsul calculat de tine pe hârtie.

**Încă NU folosim:** `string`, `struct`, matrici (`v[i][j]`), `std::vector`, recursivitate, `switch`, pointeri.

## Partea 1 — la fel ca la clasă

### 1. Cea mai lungă secvență strict crescătoare

ușor

Se dă un vector cu `d` numere întregi. Afișează cea mai lungă secvență de elemente **vecine** care cresc strict (fiecare e mai mare decât cel dinaintea lui), împreună cu lungimea ei. Dacă sunt mai multe de aceeași lungime, o afișezi pe prima.

```
v = { 4, 7, 9, 2, 3, 5, 8, 12, 6 }, d = 9
```

```
Secventa: 2 3 5 8 12
```

```
Lungime: 5
```

Exact structura din `secvente.h`: bucla exterioară caută un început, bucla interioară întinde secvența cât se poate, apoi sari cu `i` peste ce ai consumat.

### 2. Câte elemente conțin o anumită cifră

ușor

Se dau un vector cu `d` numere naturale și o cifră `c`. Afișează câte elemente ale vectorului conțin cifra `c` și care sunt acestea. Atenție: un element se numără **o singură dată**, chiar dacă are cifra `c` de mai multe ori.

```
v = { 172, 45, 717, 30, 27, 88 }, d = 6, c = 7
```

```
Contin cifra 7: 172 717 27
```

```
Total: 3
```

„O singură dată” e cheia — la 1d ai avut aceeași situație și ai rezolvat-o punând `f[cif] = 1` în loc de `f[cif]++`.

### 3. Cel mai mic număr care se repetă

ușor

Se dă un vector cu  $d$  numere naturale, toate între 1 și 100. Afișează cel mai mic număr care apare de **cel puțin două ori** în vector, împreună cu numărul lui de apariții. Dacă toate numerele sunt distincte, afișează Nu exista dublete.

```
v = { 42, 17, 8, 42, 17, 91, 17 }, d = 7  
Cel mai mic numar repetat: 17, apare de 3 ori
```

Cu un vector de frecvență, „cel mai mic care se repetă” înseamnă primul indice, luat crescător, la care  $f[i] \geq 2$ . Cât de mare trebuie să fie  $f$ ?

## Partea 2 — un pic mai grele

### 4. Cea mai lungă secvență cu aceeași sumă a cifrelor

mai greu

Se dă un vector cu  $d$  numere naturale. Afișează cea mai lungă secvență de elemente vecine care au toate **aceeași sumă a cifrelor**, împreună cu suma respectivă.

```
v = { 23, 41, 50, 14, 7, 61, 106, 90, 12 }, d = 9  
Suma cifrelor: 5  
Secventa: 23 41 50 14  
Lungime: 4
```

E aceeași formă ca exercițiul 1 — se schimbă doar întrebarea din condiția buclei interioare. Scrie `sumaCifrelor` o dată și compară rezultatele ei, nu numerele.

### 5. Cifrele comune tuturor elementelor

mai greu

Se dă un vector cu  $d$  numere naturale. Găsește cifrele care apar în **fiecare** element al vectorului și formează cu ele un număr, în ordine crescătoare. Dacă nu există nicio cifră comună, afișează 0.

```
v = { 1234, 4321, 5142, 9413 }, d = 4  
Cifre comune: 1 4  
Numarul format: 14
```

Ține un contor pe fiecare cifră: la fiecare element, adaugi 1 doar pentru cifrele prezente în el. O cifră e comună când contorul ei ajunge fix la  $d$ .

## 6. Inventarul secvențelor de numere pare

mai greu

Se dă un vector cu  $d$  numere întregi. Afișează: câte secvențe de numere pare vecine există în vector, lungimea celei mai lungi și poziția de la care începe. O secvență e „completă” — dacă ai 8 4 6, aceea e o secvență de lungime 3, nu trei secvențe de lungime 1.

```
v = { 3, 8, 4, 6, 7, 2, 5, 10, 12, 14, 16, 9 }, d = 12
```

Numar de secvente de numere pare: 3

Cea mai lunga are lungimea 4 si incepe pe pozitia 7

Atenție la numerele negative: în C++,  $-4 \% 2$  dă 0, dar  $-3 \% 2$  dă -1, nu 1. Care dintre testele  $x \% 2 == 0$  și  $x \% 2 == 1$  e cel sigur?

## Partea 3 — provocări

## 7. Cea mai lungă secvență fără elemente repetate

provocare

Se dă un vector cu  $d$  numere naturale, toate între 0 și 100. Afișează cea mai lungă secvență de elemente vecine în care **niciun număr nu se repetă**.

```
v = { 5, 2, 7, 2, 9, 4, 1, 9, 3 }, d = 9
```

Secventa: 7 2 9 4 1

Lungime: 5

Aici se întâlnesc cele două teme: o secvență ca la exercițiul 1, dar condiția de oprire se verifică cu un vector de frecvență. Partea delicată e că, la fiecare secvență nouă, frecvențele trebuie să pornească de la zero — gândește-te unde declari  $f$  ca să se întâmple asta singur.

## 8. Cea mai lungă secvență alternantă par-impar

provocare

Se dă un vector cu  $d$  numere naturale. Afișează cea mai lungă secvență de elemente vecine în care paritatea se schimbă la fiecare pas: par, impar, par, impar... (sau invers). O secvență de un singur element are lungimea 1.

```
v = { 4, 6, 3, 8, 5, 12, 7, 9, 2 }, d = 9
```

Secventa: 6 3 8 5 12 7

Lungime: 6

Nu ai nevoie de niciun vector în plus. Întrebarea e ce condiție leagă două elemente vecine — încearcă să o scrii dintr-o singură comparație, fără  $if$ -uri imbricate.

Rezolvă fiecare exercițiu într-o funcție separată și testează-o pe rând. Succes!